PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO DE JANEIRO

Alocação preditiva de recursos em nuvem via previsão de carga com Redes Neurais para minimização de custos operacionais

Gabriel Lobo de Sant'Anna Rosas

PROJETO FINAL DE GRADUAÇÃO

CENTRO TÉCNICO CIENTÍFICO - CTC

DEPARTAMENTO DE INFORMÁTICA

Curso de Graduação em Engenharia de Computação

Rio de Janeiro, Novembro de 2026

Pontifícia Universidade Católica
do Rio de Janeiro



Gabriel Lobo de Sant'Anna Rosas

Alocação preditiva de recursos em nuvem via previsão de carga com Redes Neurais para minimização de custos operacionais

Relatório de Projeto Final I, apresentado ao programa de Graduação do Departamento de Informática da PUC-Rio como requisito parcial para a obtenção do título de Engenheiro de Computação.

Orientador: Adriano Francisco Branco
Co-orientador: Jonatas dos Santos Grosman

Rio de Janeiro
Novembro de 2026

Resumo

Rosas, Gabriel Lobo de Sant'Anna. Branco, Adriano Francisco. Grosman, Jonatas dos Santos. **Alocação preditiva de recursos em nuvem via previsão de carga com Redes Neurais para minimização de custos operacionais**. Rio de Janeiro, 2026. 32 p. Relatório de Projeto Final I – Departamento de Informática. Pontifícia Universidade Católica do Rio de Janeiro.

Provedores de nuvem precisam decidir continuamente quantas instâncias manter ativas: recursos em excesso elevam o custo, e recursos insuficientes violam o acordo de nível de serviço. A alocação reativa só ajusta a capacidade depois de observar a mudança na demanda e, por isso, chega atrasada nos picos. Este trabalho investiga o quanto a qualidade da previsão de carga reduz o custo de alocação sem aumentar as violações de serviço. Para atribuir as diferenças ao previsor, a política de alocação é mantida fixa e apenas o previsor muda. São comparados métodos tradicionais de previsão (ingênuo, média móvel, sazonal ingênuo, Holt-Winters e ARIMA) e uma rede neural recorrente do tipo GRU, além de uma política reativa e do dimensionamento estático pelo pico do treino, sobre séries de utilização de processador agregadas a partir do rastro público de um *cluster* da Alibaba e avaliadas em um ambiente de simulação. As métricas são o custo normalizado pelo de um oráculo e a fração de intervalos com violação. Os resultados preliminares mostram que, com limite de 5% de intervalos com violação, ajustar a capacidade à demanda, com ou sem previsão, reduz o custo em cerca de 35% frente ao dimensionamento estático, mas que o melhor previsor fica apenas 1,7% abaixo da política reativa, diferença que uma única janela de teste não permite atribuir ao previsor, e que a GRU é competitiva com o ARIMA, sem vantagem clara. No Projeto Final II, a avaliação incorpora a memória e se estende ao rastro do Google.

Palavras-chave

Computação em nuvem; Alocação de recursos; Previsão de carga; Redes neurais; Acordo de nível de serviço.

Abstract

Rosas, Gabriel Lobo de Sant'Anna. Branco, Adriano Francisco (Advisor). Grosman, Jonatas dos Santos (Co-advisor). **Predictive cloud resource allocation via workload forecasting with neural networks for operating cost minimization**. Rio de Janeiro, 2026. 32 p. Undergraduate Thesis I – Departamento de Informática. Pontifícia Universidade Católica do Rio de Janeiro.

Cloud providers must continuously decide how many instances to keep active: excess resources raise costs, while insufficient resources violate the service level agreement. Reactive allocation adjusts capacity only after observing a change in demand and therefore lags behind peaks. This work investigates how much the quality of workload forecasting reduces allocation cost without increasing service violations. To attribute differences to the forecaster, the allocation policy is kept fixed and only the forecaster changes. Traditional forecasting methods (naive, moving average, seasonal naive, Holt-Winters and ARIMA) and a recurrent neural network of the GRU type are compared, together with a reactive policy and static sizing at the training peak, using processor utilization series aggregated from a public Alibaba cluster trace and evaluated in a simulation environment. The metrics are the cost normalized by that of an oracle and the fraction of intervals with violations. Preliminary results show that, with a 5% violation limit, adjusting capacity to demand, with or without forecasting, reduces cost by about 35% compared with static sizing, but that the best forecaster is only 1.7% cheaper than the reactive policy, a difference that a single test window cannot attribute to the forecaster, and that the GRU is competitive with ARIMA, with no clear advantage. In the second part of the project, the evaluation incorporates memory and extends to the Google trace.

Keywords

Cloud computing; Resource allocation; Workload forecasting; Neural networks; Service level agreement.

Sumário

1 Introdução **8**
1.1 Motivação 8
1.2 Definição do problema 8
1.3 Justificativa 9
1.4 Organização do documento 9

2 Situação Atual **10**
2.1 Conceitos e terminologia 10
2.2 Abordagens para previsão de carga 12
2.3 Abordagens para alocação de recursos 13
2.4 Comparativo dos trabalhos relacionados e lacuna 15
2.5 Tecnologias utilizadas 16

3 Objetivos do trabalho **17**
3.1 Solução proposta 17
3.2 Métricas de avaliação 19
3.3 Objetivos e escopo 20
3.4 Avanços em relação às soluções existentes 20
3.5 Itens a serem implementados e apresentados à banca 21

4 Revisão do Plano de Ação **22**
4.1 Atividades realizadas 22
4.2 Dificuldades encontradas 23
4.3 Resultados preliminares 23

5 Cronogramas **26**
5.1 Projeto Final I 26
5.2 Projeto Final II 26

Referências **27**

A Erro de previsão por horizonte **30**

B Parâmetros dos dados, dos previsores e do Projeto Final II **32**

Lista de figuras

Figura 3.1	Arquitetura da solução proposta	17
Figura 4.1	Demanda de processador e cobertura do rastro da Alibaba	22
Figura 4.2	Custo de cada política no ponto de operação	24

Lista de tabelas

Tabela 2.1	Comparativo entre os trabalhos relacionados	15
Tabela 2.2	Ferramentas por etapa do *pipeline*	16
Tabela 3.1	Parâmetros da alocação e da avaliação	18
Tabela 3.2	Questões de pesquisa	20
Tabela 3.3	Objetivos específicos e situação no Projeto Final I	21
Tabela 4.1	Erro das previsões no teste, com $h = 1$	23
Tabela 4.2	Pontos de operação no teste	25
Tabela 4.3	Demanda não atendida, ociosidade e alterações no teste	25
Tabela 5.1	Cronograma do Projeto Final I	26
Tabela 5.2	Cronograma do Projeto Final II	26
Tabela A.1	Erro das previsões na validação, por horizonte	30
Tabela A.2	Erro das previsões no teste, por horizonte	31
Tabela B.1	Parâmetros dos dados, dos previsores e do Projeto Final II	32

1
Introdução

1.1
Motivação

A computação em nuvem consolidou-se como a principal forma de disponibilizar infraestrutura computacional. O provedor mantém grandes centros de dados e precisa decidir, de forma contínua, quanto de processamento, memória e armazenamento deixar disponível, e, com a demanda crescente, impulsionada por aplicações de Inteligência Artificial e de análise de dados, a gestão eficiente desses recursos tornou-se central para a sustentabilidade econômica dos provedores (ROSSI et al., 2025).

A carga de trabalho, porém, não é constante: apresenta picos, padrões periódicos e variações aparentemente aleatórias. A abordagem mais comum é a alocação reativa, que ajusta os recursos depois de observar uma mudança na demanda, normalmente por regras de limiar sobre a utilização (LORIDO-BOTRAN; MIGUEL-ALONSO; LOZANO, 2014); como a reação chega atrasada, ela tende a degradar o desempenho nos picos e a desperdiçar recursos nos períodos de baixa (BODRA; KHAIRNAR, 2025). Manter recursos em excesso (superprovisionamento) garante a qualidade do serviço, mas eleva o custo; manter recursos de menos (subprovisionamento) reduz o custo, mas provoca violações dos Acordos de Nível de Serviço (*Service Level Agreements*, SLA) firmados com os clientes.

A alocação preditiva usa o histórico de uso para antecipar a demanda e provisionar recursos antes que sejam necessários (BODRA; KHAIRNAR, 2025), o que faz da previsão de carga uma pré-condição para otimizar a alocação (ROSSI et al., 2025). Essa previsão foi tradicionalmente tratada com modelos estatísticos lineares, como o modelo autorregressivo integrado de médias móveis (ARIMA) (CALHEIROS et al., 2015), e, mais recentemente, com redes neurais recorrentes, inclusive em abordagens que consideram processador e memória simultaneamente (XU et al., 2022).

1.2
Definição do problema

O problema tem duas partes encadeadas. A primeira é de previsão: dado o histórico de utilização de um *cluster*, amostrado em intervalos regulares, estimar a demanda de processador e de memória em um horizonte curto. A segunda é de decisão: a partir dessa estimativa, determinar quantas instâncias (máquinas virtuais ou contêineres) manter ativas em cada intervalo, o que pode ser formulado como a minimização do custo operacional sujeita a uma restrição de qualidade de serviço:

$$\min_{n_1,\ldots,n_T} \sum_{t=1}^{T} c \cdot n_t \quad \text{sujeito a} \quad \frac{1}{T}\sum_{t=1}^{T} \mathbf{1}\left[\, d_t > k \cdot n_t \,\right] \leq \varepsilon, \tag{1-1}$$

em que n_t é o número de instâncias ativas no intervalo t, c é o custo de uma instância por intervalo, k é a capacidade de uma instância, d_t é a demanda real, $\mathbf{1}[\cdot]$ é a função indicadora e ε é a fração máxima tolerada de intervalos com violação de SLA. Como a decisão é tomada com antecedência L (Seção 2.1.3), antes de d_t ser conhecida, ela depende de uma previsão, e a Seção 3.1 define como a restrição é verificada na prática.

1.3 Justificativa

A relevância do problema é econômica e operacional. Cada ponto percentual de recursos ociosos representa custo para o provedor, e cada violação de SLA representa perda de qualidade percebida pelo cliente e, eventualmente, penalidades contratuais. Apesar do volume de trabalhos sobre previsão de carga, nos trabalhos revisados falta uma avaliação de ponta a ponta, com dados públicos, que meça quanto a previsão reduz o custo de alocação quando redes neurais e métodos tradicionais são submetidos à mesma política, lacuna discutida na Seção 2.4.

1.4 Organização do documento

O Capítulo 2 apresenta os conceitos, as abordagens existentes e as tecnologias utilizadas; o Capítulo 3, a solução proposta, as métricas, os objetivos e o escopo; o Capítulo 4, a revisão do plano de ação e os resultados preliminares; e o Capítulo 5, os cronogramas do Projeto Final I (PF I) e do Projeto Final II (PF II).

2
Situação Atual

Este capítulo define a terminologia (Seção 2.1), descreve as abordagens existentes para previsão de carga e para alocação de recursos, com suas limitações (Seções 2.2 e 2.3), compara os trabalhos mais próximos desta proposta (Seção 2.4) e apresenta as tecnologias utilizadas (Seção 2.5).

2.1
Conceitos e terminologia

2.1.1
Elasticidade, carga de trabalho e séries temporais

Uma das características essenciais da computação em nuvem é a elasticidade rápida, isto é, a capacidade de provisionar e liberar recursos, em alguns casos de forma automática, acompanhando a demanda (MELL; GRANCE, 2011). A carga de trabalho (*workload*) de um *cluster*, formada por tarefas em lote, serviços de longa duração e requisições de usuários, se manifesta como utilização de recursos, principalmente processador e memória. Agregada em intervalos regulares de duração Δ, ela forma uma série temporal $y_1, \ldots, y_t$, que pode combinar tendência, sazonalidade e ruído (HYNDMAN; ATHANASOPOULOS, 2018) e, nas cargas de nuvem, muda bruscamente em pouco tempo (XU et al., 2022). Prever a carga significa estimar $\hat{y}_{t+h}$, o valor da série h intervalos à frente (horizonte de previsão), a partir das w observações mais recentes (janela de entrada). O intervalo em que a previsão é feita é a origem τ: uma previsão feita em τ só pode usar observações até τ. A previsão é univariada quando usa apenas a própria série e multivariada quando incorpora outras, como a memória na previsão do processador (XU et al., 2022).

Entre os rastros (*traces*) públicos de *clusters* reais estão o do Google, com oito células do gerenciador Borg em maio de 2019 (WILKES, 2020), e o da Alibaba de 2018, com cerca de 4.000 máquinas ao longo de oito dias (ALIBABA GROUP, 2018); transformá-los em séries regulares faz parte deste trabalho. Intervalos de cinco minutos são os usuais com esses rastros (ROSSI et al., 2025; AGULLÓ et al., 2025); Rossi et al. (2025) preveem a demanda dez minutos à frente, horizonte que consideram suficiente para a maioria das aplicações de alocação.

2.1.2
Acordos de nível de serviço

A qualidade de um serviço em nuvem é expressa pelo indicador de nível de serviço (*Service Level Indicator*, SLI), uma medida quantitativa como a latência, pelo objetivo de nível de serviço (*Service Level Objective*, SLO), um valor-alvo para um SLI, e pelo SLA, que formaliza esses objetivos em contrato e define as

consequências, geralmente financeiras, de não cumpri-los (BEYER et al., 2016). Como os rastros públicos não registram a latência percebida pelos usuários, a literatura de alocação aproxima a violação de SLA por medidas derivadas da utilização, como a fração do tempo em que os servidores operam com utilização máxima (BELOGLAZOV; BUYYA, 2012). Este trabalho adota uma aproximação do mesmo tipo: o SLI é medido por intervalo, atendido quando $d_t \leq k \cdot n_t$ na notação da Equação 1-1, e o SLO é que no máximo uma fração ε dos intervalos fique sem atendimento, que é a restrição daquela equação. A simplificação não captura filas nem a degradação gradual perto da saturação, mas permite comparar estratégias de forma consistente.

2.1.3 Provisionamento reativo e preditivo

O escalonamento automático (*autoscaling*) ajusta os recursos de uma aplicação sem intervenção humana; este trabalho trata do escalonamento horizontal, que altera o número de instâncias. Entre as famílias de técnicas classificadas por Lorido-Botran, Miguel-Alonso e Lozano (2014), a distinção central aqui é entre o provisionamento reativo, que decide a partir do que já foi observado, normalmente a utilização corrente, e o preditivo, que decide a partir de uma estimativa $\hat{d}_{t+h}$ da demanda futura (a série prevista é a própria demanda, $y_t = d_t$) e já é oferecido por provedores comerciais (AMAZON WEB SERVICES, 2024). No extremo oposto, sem escalonamento, está o dimensionamento estático pelo pico, que evita violações enquanto a demanda não passa desse pico, mas mantém capacidade ociosa no resto do tempo; frente a ele, a alocação guiada por ARIMA de Calheiros et al. (2015) economizou de 16,6% a 34,1% das horas de máquina virtual.

Como uma nova instância leva um tempo para ficar disponível, o atraso de inicialização (MAO; HUMPHREY, 2012), a decisão tomada com as observações até o intervalo τ só passa a valer em $\tau + L$, em que L é a antecedência da decisão (*lead time*), igual a um intervalo mais o atraso de inicialização; sem atraso, $L = 1$. A política reativa decide em τ com a utilização observada em τ, e a preditiva tenta antecipar a demanda de $\tau + L$. Em cada intervalo, a decisão pode errar em duas direções: com $k \cdot n_t > d_t$, a capacidade ociosa $k \cdot n_t - d_t$ é custo sem retorno (superprovisionamento); com $k \cdot n_t < d_t$, parte da demanda não é atendida (subprovisionamento).

Essa assimetria separa dois tipos de medida. As métricas de previsão, como a raiz do erro quadrático médio (RMSE) e o erro absoluto médio (MAE), medem a distância entre a previsão e o valor real e são simétricas: tratam da mesma forma a superestimação, que gera custo, e a subestimação, que gera violações. As métricas de alocação medem o resultado das decisões, como o custo e a fração de intervalos com violação. Por isso, um modelo com menor erro não leva necessariamente às melhores decisões; as definições adotadas neste trabalho estão na Seção 3.2.

2.2 Abordagens para previsão de carga

2.2.1 Modelos estatísticos

Os métodos mais simples servem de referência para todos os demais (HYNDMAN; ATHANASOPOULOS, 2018). O ingênuo, ou de persistência, repete o último valor observado, $\hat{y}_{t+h} = y_t$; a média móvel usa a média das w últimas observações; e o sazonal ingênuo repete o valor de um período sazonal antes, $\hat{y}_{t+h} = y_{t+h-s}$, com o período s medido em intervalos e $h \leq s$.

Dois modelos estimam parâmetros a partir do histórico. O Holt-Winters é uma suavização exponencial com componentes de tendência e de sazonalidade, adequada aos ciclos diários das cargas de nuvem (HYNDMAN; ATHANASOPOULOS, 2018). O ARIMA (*AutoRegressive Integrated Moving Average*) (BOX et al., 2015) diferencia a série d vezes para torná-la estacionária e a descreve como combinação linear de p valores e de q erros de previsão passados.

Calheiros et al. (2015) usaram um ARIMA para provisionar máquinas virtuais a partir da demanda horária de requisições à Wikipédia, simulada no CloudSim (CALHEIROS et al., 2011). Provisionar pelo limite superior do intervalo de confiança, em vez do valor previsto, reduziu as violações de qualidade de serviço (QoS) ao custo de mais horas de máquina, um ajuste feito depois da previsão que controla o equilíbrio entre custo e violações. A avaliação, contudo, envolve um único previsor.

Limitações. Os modelos estatísticos assumem relações lineares e, no ARIMA, estacionariedade após a diferenciação, hipóteses que as mudanças de regime e os picos das cargas de nuvem violam; em sua forma básica, também são univariados. Por outro lado, exigem poucos dados, são baratos de ajustar e interpretáveis, e por isso devem estar em qualquer comparação: uma rede neural só se justifica se os superar sob as mesmas condições.

2.2.2 Redes neurais recorrentes

Para aplicar uma rede neural, a série é transformada em um problema supervisionado por janela deslizante, com as w observações mais recentes na entrada; para vários horizontes, a estratégia direta usa uma saída por horizonte, em vez de realimentar as próprias previsões e acumular erro (BEN TAIEB et al., 2012). As redes neurais recorrentes (RNN) leem a janela em ordem e mantêm um estado interno que funciona como memória, mas têm dificuldade de aprender relações entre observações distantes (BENGIO; SIMARD; FRASCONI, 1994). A LSTM (*Long Short-Term Memory*) resolve esse problema com portas que aprendem o que guardar e o que esquecer (HOCHREITER; SCHMIDHUBER, 1997). A GRU (*Gated Recurrent Unit*) simplifica essa ideia com duas portas (CHO et al., 2014), o que reduz o número de parâmetros, com desempenho comparável

ao da LSTM (CHUNG et al., 2014). O esDNN (XU et al., 2022) combina uma GRU modificada com camadas convolucionais para prever processador e memória nos rastros da Alibaba e do Google; a GRU pura é competitiva em horizontes curtos, mas a avaliação da previsão usa apenas métricas de erro simétricas.

Limitações. Redes recorrentes exigem mais dados e mais ajuste do que os modelos estatísticos e, treinadas para minimizar um erro médio, tendem a suavizar os picos de demanda. Agulló et al. (2025) mostram que previsores tradicionais e profundos falham justamente nesses picos, sem que as métricas usuais revelem a falha, e que ajustar a saída da previsão melhora a antecipação dos picos ao custo de mais superprovisionamento. Isso reforça que o erro médio não basta para avaliar um previsor usado na alocação.

2.2.3 Modelos com quantificação de incerteza

Rossi et al. (2025) propõem uma rede neural bayesiana (HBNN) e uma LSTM com saída distribucional (LSTMD), que preveem uma distribuição para a demanda de processador e de memória nos rastros do Google e da Alibaba, e mostram que métricas de erro pontual não distinguem bem os modelos, ao contrário de métricas de nível de serviço calculadas sobre o limite superior do intervalo de confiança: a taxa de sucesso, fração dos intervalos em que a demanda fica abaixo desse limite, e o total de recursos previstos, soma dos limites em relação à demanda. Os autores também concluem que a transferência de aprendizado funciona dentro de um mesmo provedor, mas se degrada entre provedores diferentes, o que motiva verificar se as conclusões obtidas em um rastro se mantêm no outro (Seção 3.3).

Limitações. Essas métricas são calculadas sobre a própria previsão, sem mecanismo de alocação, integração que os autores deixam como trabalho futuro, e supõem capacidade contínua, e não instâncias discretas com atraso de inicialização. Este trabalho mede diretamente as métricas de alocação e mantém a ideia de provisionar pelo limite superior na forma determinística da margem m (Seção 2.3.2).

2.3 Abordagens para alocação de recursos

2.3.1 Heurísticas e políticas reativas

As políticas baseadas em limiares são a forma mais difundida de escalonamento automático (LORIDO-BOTRAN; MIGUEL-ALONSO; LOZANO, 2014). Se a utilização média das instâncias permanece acima de um limiar superior por alguns intervalos, uma instância é adicionada, e, se permanece abaixo de um limiar inferior, uma é removida. Um período de espera (*cooldown*) após cada ação impede oscilações. O *Horizontal Pod Autoscaler* (HPA) do Kubernetes usa uma variação

proporcional, que calcula o número de réplicas pela razão entre a métrica observada u_t e um valor-alvo u^* (THE KUBERNETES AUTHORS, 2024):

$$n_{t+1} = \left\lceil n_t \cdot \frac{u_t}{u^*} \right\rceil , \tag{2-1}$$

com uma faixa de tolerância, dentro da qual nenhuma ação é tomada, e uma janela de estabilização para reduções, por padrão de 10% e de 5 minutos. O esDNN usa como referência outra política reativa, que mantém ativa a média do número de máquinas dos cinco intervalos anteriores (XU et al., 2022).

Limitações. Por construção, as políticas reativas só respondem depois da mudança de carga, defasagem que o atraso de inicialização aumenta. Os limiares precisam ser ajustados para cada aplicação: limiares agressivos causam oscilações, e conservadores mantêm capacidade ociosa. Ainda assim, por serem simples, previsíveis e amplamente usadas, a variante proporcional do HPA (Equação 2-1) é a referência reativa deste trabalho.

2.3.2 Alocação guiada por previsão

Na alocação guiada por previsão, a decisão tem duas etapas desacopladas: um modelo estima a demanda $\hat{d}_{t+h}$ do intervalo em que a decisão terá efeito, e uma regra converte essa estimativa em número de instâncias, normalmente dividindo-a pela capacidade de uma instância e acrescentando uma margem de segurança $m \geq 0$, com um piso de $n_{\min}$ instâncias:

$$n_{t+h} = \max\left(n_{\min}, \left\lceil \frac{(1+m)\,\hat{d}_{t+h}}{k} \right\rceil\right). \tag{2-2}$$

Na alocação, t é a origem τ e o horizonte é a antecedência, $h = L$, para que a previsão se refira ao intervalo em que a decisão vale (Seção 2.1.3). Margens maiores reduzem as violações e aumentam o custo, e escolher a margem em um conjunto de validação é a forma prática de buscar o menor custo que respeita o limite ε da Equação 1-1. Com um modelo que fornece uma distribuição, a margem pode dar lugar a um quantil (ROSSI et al., 2025; CALHEIROS et al., 2015).

O MagicScaler (PAN et al., 2023) combina uma previsão com incerteza a uma otimização estocástica que minimiza o custo sujeito a um limite de demanda não atendida e, em *clusters* da Alibaba Cloud, reduziu a perda de qualidade de serviço em relação a uma política reativa, com pequeno aumento de custo. A vantagem do desacoplamento é a modularidade: com a regra de decisão fixa, é possível trocar apenas o previsor e atribuir a ele as diferenças de custo e de violações. A alternativa acoplada é o aprendizado por reforço, em que um agente aprende a mapear o estado do sistema diretamente em ações, guiado por uma recompensa que penaliza custo e violações (LORIDO-BOTRAN; MIGUEL-ALONSO; LOZANO, 2014).

Limitações. A decisão herda o erro da previsão, e uma margem fixa aplica a mesma folga a todos os intervalos, calibrada em uma validação que pode não

representar o período avaliado. Nos trabalhos revisados, a regra é aplicada a um único previsor, e o MagicScaler usa dados privados. Na alternativa acoplada, os métodos são sensíveis ao projeto da recompensa e aos hiperparâmetros (BODRA; KHAIRNAR, 2025), e, quando a política aprendida decide mal, não se sabe se o problema está na antecipação da demanda ou na decisão. Por isso, este trabalho mantém a regra desacoplada e fixa, e a comparação com uma política aprendida fica para trabalhos futuros.

2.4 Comparativo dos trabalhos relacionados e lacuna

Tabela 2.1 – Comparativo entre os trabalhos relacionados

Trabalho	**Técnica de previsão**	**Dados**	**Métricas**	**Acopla à alocação**	**Mesma política**
Calheiros et al. (2015)	ARIMA	Wikipédia (1 h)	Precisão, QoS e horas de máquina	Sim	Não
Xu et al. (2022)	GRU com convolução, multivariada	Alibaba e Google	Erro e máquinas ativas	Parcial	Não
Pan et al. (2023)	Atenção e processo gaussiano	Alibaba Cloud (privados)	Erro, custo e perda de QoS	Sim	Não
Rossi et al. (2025)	Bayesianas (HBNN, LSTMD)	Alibaba e Google	Erro e nível de serviço	Não	–
Agulló et al. (2025)	Vários, com saída ajustada	Alibaba e Google	Erro, picos e super-provisionamento	Não	–
Bodra e Khairnar (2025)	Revisão de vários métodos	Variados	Heterogêneas	–	–
Este trabalho	GRU e cinco métodos tradicionais	Alibaba (PF I); Google (PF II)	Erro, custo e violações	Sim	Sim

Fonte: Elaborado pelo autor.

Nota: o traço indica que o critério não se aplica.

A Tabela 2.1 resume os trabalhos mais próximos desta proposta, indicando também se a previsão é acoplada a uma decisão de alocação e se diferentes previsores são comparados sob a mesma política. Esses trabalhos formam duas linhas que não se encontram. Os de previsão (XU et al., 2022; ROSSI et al., 2025; AGULLÓ et al., 2025) comparam vários modelos por métricas calculadas sobre a própria previsão, sem decisão de alocação, e as de erro pontual não dão um quadro claro da utilidade dos modelos (ROSSI et al., 2025) nem revelam as falhas nos picos (AGULLÓ et al., 2025). Os que acoplam a previsão à alocação aplicam a decisão a um único previsor, comparado com a alocação estática pelo pico (CALHEIROS et al., 2015) ou com uma política reativa (PAN et al., 2023), e o esDNN compara sua simulação de escalonamento apenas com uma política baseada na média das observações anteriores (XU et al., 2022). A revisão de

Bodra e Khairnar (2025) observa, por fim, que a literatura carece de comparações quantitativas sistemáticas.

A lacuna que este trabalho busca preencher, até onde se verificou nos trabalhos revisados, é, portanto, a de uma avaliação de ponta a ponta, com dados públicos de um *cluster* real, que submeta previsores de complexidade crescente, do método ingênuo à rede neural, à mesma política de alocação, calibrada para o mesmo limite de violações ε, e meça o custo e as violações resultantes. A pergunta deixa de ser qual modelo erra menos e passa a ser quanto custo cada modelo economiza para um mesmo nível de serviço.

2.5 Tecnologias utilizadas

O protótipo é escrito em Python 3.12, pela disponibilidade de bibliotecas para séries temporais e aprendizado profundo, e o código está em repositório público (ROSAS, 2026). A Tabela 2.2 relaciona as ferramentas usadas em cada etapa do *pipeline*.

Tabela 2.2 – Ferramentas por etapa do *pipeline*

Etapa	**Ferramenta**	**Uso no trabalho**
Dados	DuckDB (RAASVELDT; MÜHLEISEN, 2019)	Agregação do rastro bruto por consultas SQL, sem carregá-lo inteiro na memória
	Parquet, pandas (MCKINNEY, 2010) e NumPy (HARRIS et al., 2020)	Armazenamento e manipulação das séries; previsores ingênuo, média móvel e sazonal ingênuo
Previsão	statsmodels (SEABOLD; PERKTOLD, 2010); PyTorch (PASZKE et al., 2019) para CPU	ARIMA e Holt-Winters; GRU, sem exigir GPU
Simulação	Python; CloudSim Plus (SILVA FILHO et al., 2017) como validação opcional (PF II)	Alocação intervalo a intervalo e métricas
Resultados	Matplotlib (HUNTER, 2007)	Figuras
Ambiente	uv, Docker, Makefile, pytest, Git e GitHub	Dependências travadas, contêiner, encadeamento das etapas, testes e versionamento

Fonte: Elaborado pelo autor.

O CloudSim (CALHEIROS et al., 2011), de que deriva o CloudSim Plus, é um simulador, em Java, de eventos discretos que modela centros de dados, servidores, máquinas virtuais e a execução de tarefas, usado, por exemplo, na avaliação do provisionamento guiado por ARIMA de Calheiros et al. (2015). Para a reprodutibilidade, as versões das dependências ficam travadas, todos os parâmetros dos experimentos ficam em um arquivo de configuração, e nenhum no código, e um único comando regenera todas as tabelas e figuras a partir dos dados brutos.

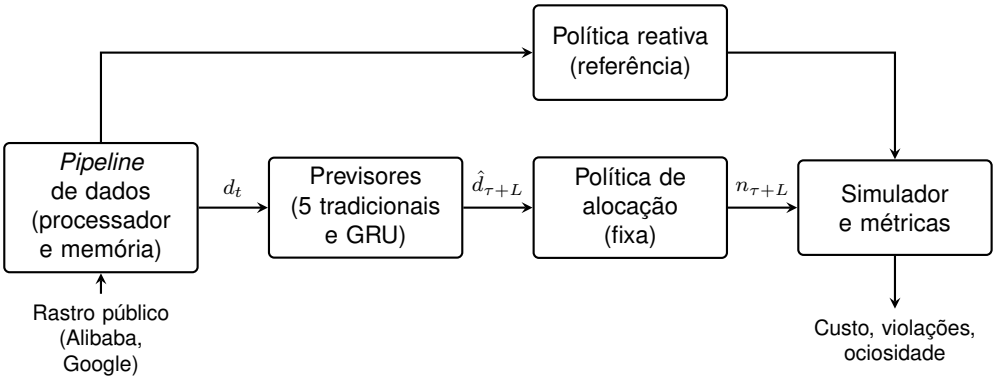
3
Objetivos do trabalho

Este capítulo descreve a solução proposta para a lacuna da Seção 2.4, as métricas de avaliação, os objetivos, os avanços em relação às soluções existentes e os itens a apresentar à banca.

3.1
Solução proposta

A solução proposta é um ambiente de avaliação da alocação preditiva, que mede quanto custo operacional uma previsão de carga melhor permite economizar, para um mesmo nível de serviço. Ele segue a abordagem desacoplada da Seção 2.3.2, com os quatro componentes da Figura 3.1, e mantém a política de alocação fixa, variando apenas o previsor: a regra e o procedimento de calibração da margem são os mesmos para todos, e só o valor da margem depende do previsor. Assim, diferenças de custo ou de violações entre duas configurações podem ser atribuídas à previsão, e não à regra de decisão.

Figura 3.1 – Arquitetura da solução proposta



Fonte: Elaborado pelo autor.

***Pipeline* de dados.** Converte um rastro público de *cluster* em séries de demanda de processador e de memória com intervalos de cinco minutos, expressas em máquinas equivalentes totalmente utilizadas e divididas cronologicamente em treino, validação e teste.

Previsores. Os cinco métodos tradicionais da Seção 2.2.1 e uma GRU que, a partir das últimas seis horas, prevê todos os horizontes de uma vez (estratégia direta). Todos usam uma interface comum, que só dá acesso ao histórico até a origem τ. A parada antecipada (*early stopping*) da GRU usa o fim do treino, e a validação fica reservada à escolha dos parâmetros das políticas.

Política de alocação. Converte a demanda prevista com $h = L$ em instâncias pela Equação 2-2, com margem m. As referências são a política reativa do HPA (Equação 2-1), com utilização-alvo u^*; o dimensionamento estático, que mantém as n_{pico} instâncias necessárias no pico do treino; e o oráculo (Seção 3.2). Na

reativa, $u_\tau = d_\tau/(k \cdot n_\tau)$, e a Equação 2-1 recomenda $\lceil d_\tau/(k \cdot u^*) \rceil$ instâncias: sem a tolerância e a janela de estabilização, ela seria a preditiva com o previsor ingênuo e $m = 1/u^* - 1$. A comparação com a reativa muda, portanto, também a regra de decisão.

Simulador e métricas. Percorre o período avaliado de cinco em cinco minutos e mantém ativas, em cada intervalo, as instâncias decididas pela política, que só passam a valer depois do tempo de inicialização (Seção 2.1.3). Em cada intervalo, compara a capacidade ativa com a demanda real e, ao fim, calcula o custo, que conta só as instâncias ativas, as violações e as demais métricas da Seção 3.2. É implementado em Python; se o cronograma permitir, parte dos experimentos será reproduzida no CloudSim Plus (SILVA FILHO et al., 2017) como validação.

Os parâmetros das políticas, a margem m na preditiva e a utilização-alvo u^* na reativa, são escolhidos na validação e avaliados no teste. Para cada limite de violações ε, escolhe-se, na grade da Tabela 3.1, o valor mais barato entre os que respeitam o limite na validação; o resultado desse valor no teste é o **ponto de operação** da política. Como o teste não participa da escolha, o ponto de operação pode ultrapassar ε no teste. O dimensionamento estático e o oráculo não têm parâmetro.

A Tabela 3.1 reúne os valores usados no protótipo, mantidos no Projeto Final II salvo na sensibilidade a L, nas várias janelas de teste e no rastro do Google (OE7 a OE9, Tabela 3.3). Os parâmetros dos previsores, as janelas de teste e as sementes estão na Tabela B.1, no Apêndice B.

Tabela 3.1 – Parâmetros da alocação e da avaliação

Parâmetro	**Símbolo**	**Valor**
Intervalo de agregação	Δ	5 minutos
Divisão treino/validação/teste (PF I)	–	60% / 20% / 20%, cronológica
Horizontes de previsão avaliados	h	1, 2, 3 e 6 intervalos (5 a 30 min)
Antecedência da decisão	L	1 intervalo; 2 na análise de sensibilidade
Instâncias no pico do treino	n_{pico}	50 (cada instância vale 2% do pico)
Capacidade de uma instância	k	Pico do treino dividido por n_{pico} (60,94 máquinas equivalentes)
Custo por instância e intervalo	c	1
Número mínimo de instâncias	$n_{\min}$	1
Grade de margens (preditiva)	m	0 a 0,60, com passo 0,01
Grade de utilizações-alvo (reativa)	u^*	0,50 a 0,99, com passo 0,01
Tolerância e estabilização (reativa)	–	10% e 6 intervalos (30 min)
Limites de violação	ε	1% e 5%

Fonte: Elaborado pelo autor.

3.2 Métricas de avaliação

Métricas de previsão. Medem o quanto a previsão se aproxima do valor real, independentemente de como ela será usada. Para N previsões $\hat{y}_i$ e valores reais y_i, as mais comuns são o erro quadrático médio (MSE), sua raiz (RMSE), o erro absoluto médio (MAE) e o erro percentual absoluto médio (MAPE), às quais este trabalho acrescenta o viés, que é o erro médio com sinal:

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2, \qquad \mathrm{RMSE} = \sqrt{\mathrm{MSE}}, \qquad \mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|,$$
$$\mathrm{MAPE} = \frac{100\%}{N}\sum_{i=1}^{N}\left|\frac{y_i - \hat{y}_i}{y_i}\right|, \qquad \text{viés} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i). \tag{3-1}$$

Como o MAPE é instável com y_i próximo de zero, ele é calculado só sobre os demais pontos, e a fração mantida (cobertura da máscara) é informada. Como as quatro primeiras métricas são simétricas e a alocação não é (Seção 2.1.3), são usados também o viés, cujo valor negativo indica tendência a subestimar a demanda, e a taxa de subestimação, que é a fração das previsões abaixo do valor real.

Métricas de alocação. Medem o resultado das decisões tomadas a partir da previsão. Para T intervalos:

$$C = \sum_{t=1}^{T} c \cdot n_t, \qquad V = \frac{1}{T}\sum_{t=1}^{T}\mathbf{1}\left[\, d_t > k \cdot n_t \,\right],$$
$$U = \frac{\sum_t \max(0,\, d_t - k \cdot n_t)}{\sum_t d_t}, \qquad O = \frac{\sum_t \max(0,\, k \cdot n_t - d_t)}{\sum_t k \cdot n_t},$$
$$A = \sum_{t=2}^{T}\mathbf{1}\left[\, n_t \neq n_{t-1} \,\right], \qquad C_{\mathrm{norm}} = \frac{C}{C_{\text{oráculo}}}, \tag{3-2}$$

em que C é o custo total, V é a fração de intervalos com violação, U é a fração da demanda não atendida, O é a fração da capacidade ociosa e A é o número de alterações em n_t, indesejáveis na prática. U mede a magnitude das violações, que V apenas conta, e O mede a capacidade paga e não usada. C e V são, respectivamente, a função objetivo e o lado esquerdo da restrição da Equação 1-1. O custo é normalizado pelo de um oráculo, que conhece a demanda futura e aloca, em cada intervalo, o menor número de instâncias que a atende, $\lceil d_t/k \rceil$. Como nenhuma alocação mais barata atende toda a demanda, $C_{\text{oráculo}}$ é o piso do custo sem violações; uma política que viola pode custar menos que o oráculo, e por isso C_{norm} deve ser lido sempre junto com V.

3.3 Objetivos e escopo

O objetivo geral é avaliar, em um ambiente de simulação alimentado por rastros públicos, a redução de custo obtida pela alocação guiada por uma rede neural recorrente, em comparação com métodos tradicionais sob a mesma política e com uma política reativa, para limites fixos de violação de SLA.

O objetivo geral se desdobra nas cinco questões de pesquisa (QP) da Tabela 3.2.

Tabela 3.2 – Questões de pesquisa

QP	Questão
QP1	Para um mesmo limite de violações ε, a alocação guiada pela rede neural tem custo menor do que a guiada pelos métodos tradicionais? De quanto é a diferença?
QP2	Um previsor com menor erro (RMSE, MAE) leva necessariamente a uma alocação mais barata no ponto de operação?
QP3	Qual é o ganho da alocação preditiva em relação à reativa, e como ele varia com a antecedência da decisão L?
QP4	As conclusões obtidas no rastro da Alibaba se mantêm no rastro do Google?
QP5	Incluir a memória como entrada da GRU reduz o custo de alocação em relação à GRU univariada, para o mesmo ε?

Fonte: Elaborado pelo autor.

A Tabela 3.3 lista os objetivos específicos (OE), as questões de pesquisa a que respondem e a situação de cada um ao fim do Projeto Final I. Com as várias janelas e sementes de OE8, um previsor só será considerado melhor que outro se o intervalo de 95% da diferença pareada de C_{norm}, obtido por *bootstrap* (EFRON; TIBSHIRANI, 1993), excluir zero, com correção de Holm (HOLM, 1979); o erro de previsão será comparado pelo teste de Diebold-Mariano (DIEBOLD; MARIANO, 1995).

Ficam fora do escopo a implantação em uma nuvem real, o posicionamento e a migração de máquinas virtuais, o consumo de energia, os múltiplos provedores e o aprendizado por reforço para a decisão. O simulador trata a capacidade de forma agregada, sem posicionamento de máquinas virtuais, e, no protótipo do Projeto Final I, a alocação considera só o processador, com a GRU univariada.

3.4 Avanços em relação às soluções existentes

Sem propor uma nova arquitetura, já que a GRU é deliberadamente simples, o trabalho busca quatro avanços, cada um em resposta a uma limitação apontada no Capítulo 2 e ligado aos objetivos que o realizam:

- **Comparação controlada:** do ingênuo à rede neural, sob a mesma política, enquanto os trabalhos revisados aplicam a decisão a um único previsor (Seção 2.4); OE2, OE3 e OE5.

Tabela 3.3 – Objetivos específicos e situação no Projeto Final I

OE	Descrição	QP	Situação no PF I
OE1	**Dados:** agregar o rastro da Alibaba de 2018 em séries de 5 min, com lacunas tratadas e relatório de qualidade	–	Implementado
OE2	**Previsores:** implementar os seis previsores e avaliar seu erro em cada horizonte	–	Implementado
OE3	**Simulador e políticas:** implementar e testar o simulador e as políticas da Seção 3.1	–	Protótipo em Python implementado
OE4	**Memória:** incluir a memória como segundo recurso na alocação, com n_t dado pelo maior entre os necessários para processador e para memória, e como entrada da GRU, comparada à GRU univariada	QP5	Previsto para o PF II
OE5	**Custo e violações:** pontos de operação para $\varepsilon = 1\%$ e 5% e curvas de custo contra violações	QP1, QP3	Preliminar
OE6	**Erro e custo:** confrontar o erro com $h = L$ e o custo no ponto de operação	QP2	Preliminar
OE7	**Sensibilidade:** efeito da antecedência L sobre os pontos de operação	QP3	Parcial ($L = 1$ e $L = 2$)
OE8	**Robustez:** várias sementes da GRU e várias janelas de teste, com média e dispersão	–	Previsto para o PF II
OE9	**Replicação:** mesmo protocolo no rastro do Google de 2019, mudando só a leitura dos dados	QP4	Previsto para o PF II
OE10	**Reprodutibilidade:** repositório público que regenera tabelas e figuras com um comando	–	Implementado; a ser mantido

Fonte: Elaborado pelo autor.

- **Custo a um mesmo nível de serviço:** instâncias discretas e antecedência, em vez do nível de serviço medido sobre a própria previsão, com capacidade contínua (Seção 2.2.3); OE5 e OE7.
- **Erro contra custo:** a QP2 testa se menor erro implica menor custo, o que a comparação por métricas de erro simétricas não mostra (Seção 2.2.2); OE6.
- **Dados públicos e reprodutibilidade:** dois rastros públicos e resultados regeneráveis, em contraste com avaliações sobre dados privados (Seção 2.3.2); OE1, OE9 e OE10.

3.5 Itens a serem implementados e apresentados à banca

Os itens a implementar são os da Tabela 3.3, mais o relatório final e a apresentação. Na defesa, serão apresentados: (i) as curvas de custo contra violações e os pontos de operação de todos os previsores e da política reativa, nos dois rastros; (ii) a comparação entre a ordenação dos previsores por erro e por custo; (iii) a análise de sensibilidade à antecedência da decisão L; e (iv) a regeneração dos resultados a partir dos dados brutos pelo repositório.

Se o cronograma permitir, a margem fixa poderá ser trocada por um quantil previsto pela rede (ROSSI et al., 2025; PAN et al., 2023).

4
Revisão do Plano de Ação

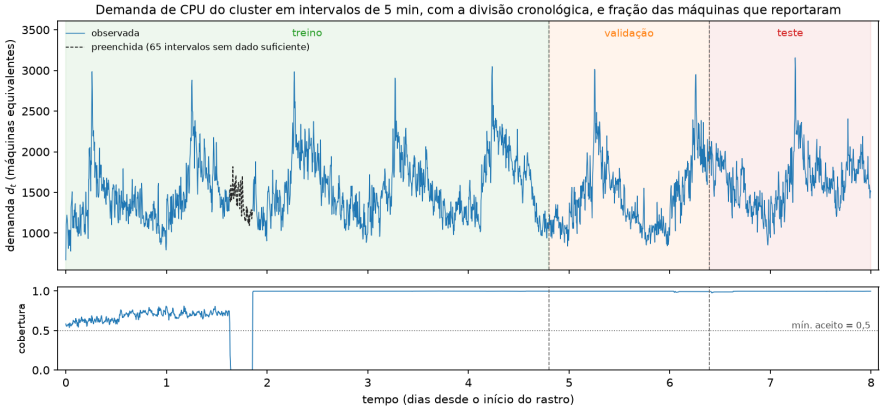
Este capítulo compara o plano do Projeto Final I com o que foi realizado, discute as dificuldades e apresenta os resultados preliminares, que sustentam a viabilidade da proposta. A proposta previa a avaliação no CloudSim Plus, mas o protótipo usou um simulador próprio, em Python, porque a decisão estudada se resume a escolher n_t a partir da demanda agregada e o simulador pôde ser coberto por testes; ele continua no Projeto Final II, com o CloudSim Plus como validação opcional. Ficam para o Projeto Final II os objetivos OE4, OE8 e OE9 e a conclusão de OE5 a OE7 (Tabela 5.2).

4.1
Atividades realizadas

A meta do Projeto Final I foi um protótipo de ponta a ponta, do rastro bruto da Alibaba ao custo e às violações de cada política, como evidência de viabilidade. A construção foi dividida em fases curtas, cada uma com uma especificação e um critério de aceite, todas concluídas antes do prazo de 31/10/2026; as datas, tiradas do histórico de versões do repositório (ROSAS, 2026), estão na Tabela 5.1.

O rastro da Alibaba foi agregado em uma série de demanda de processador com intervalos de cinco minutos, que cobre os oito dias do rastro. Como nem todas as máquinas enviam medições em todos os intervalos, a demanda é a utilização média das máquinas que reportaram, estendida ao total de máquinas, o que evita subestimá-la nos intervalos incompletos. As lacunas curtas foram interpoladas, e a interrupção de cerca de cinco horas na coleta, dentro do treino, foi preenchida com o mesmo horário do dia anterior. A Figura 4.1 mostra a série, a divisão cronológica, o trecho preenchido, em tracejado, e a cobertura, com o limite abaixo do qual o intervalo é lacuna. A validação e o teste têm cobertura quase completa, e o trecho menos confiável, no início do rastro, fica no treino.

Figura 4.1 – Demanda de processador e cobertura do rastro da Alibaba



Fonte: Elaborado pelo autor.

Os seis previsores foram implementados sob a interface comum da Seção 3.1 e avaliados com origem móvel (*walk-forward*); testes automatizados verificam que alterar os dados posteriores à origem não muda nenhuma previsão. O simulador e as políticas têm testes de casos calculados à mão e verificações de sanidade, como a de que o oráculo não viola nenhum intervalo.

4.2 Dificuldades encontradas

Holt-Winters. O statsmodels não aplica os parâmetros estimados a dados novos, e reajustar o modelo em cada origem seria lento demais. A recursão do método foi implementada no próprio previsor, com os parâmetros do treino, e um teste confirma que ela reproduz a biblioteca até a precisão de máquina.

Ponto flutuante. Três problemas tiveram origem no ponto flutuante: o oráculo aparecia com violações inexistentes, porque $k \cdot \lceil d_t/k \rceil$ pode ficar ligeiramente abaixo de d_t; o arredondamento para cima saltava para o inteiro seguinte quando o valor exato era inteiro; e a soma paralela do DuckDB variava na última casa decimal entre execuções. Comparações e arredondamentos passaram a usar tolerância relativa de 10^{-9}, e a série, seis casas decimais.

Tamanho do teste. Com 462 intervalos, um intervalo com violação vale 0,22 ponto percentual de V, e a diferença entre cumprir ou não $\varepsilon = 1\%$ no teste é de um único intervalo. A primeira versão descartava o início do rastro, de cobertura menor; usar o rastro completo aumentou o teste em 30%, mas não mudou a ordem de grandeza do problema, o que motiva a avaliação com várias janelas de teste (OE8).

4.3 Resultados preliminares

Os resultados, com os parâmetros da Tabela 3.1, são preliminares: com uma única janela de teste e uma única semente da GRU, diferenças de poucos por cento entre previsores não permitem afirmar que um é melhor que outro.

No erro de previsão (Tabela 4.1, em máquinas equivalentes), o ARIMA e a GRU são os melhores, e o sazonal ingênuo, que repete o valor de um dia antes, é o pior por larga margem; o erro nos demais horizontes está no Apêndice A.

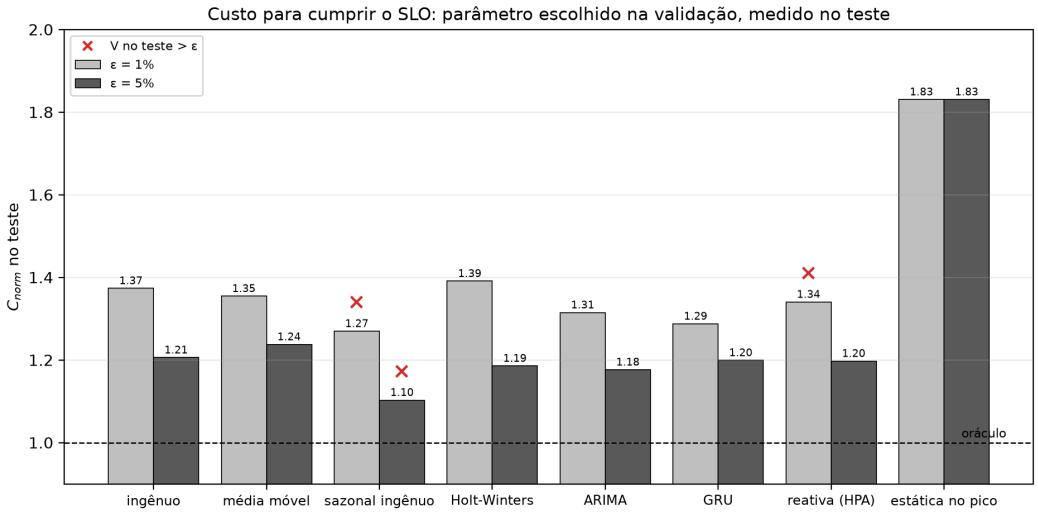
Tabela 4.1 – Erro das previsões no teste, com $h = 1$

Previsor	RMSE	MAE	Viés	Taxa de subestimação
ARIMA	157,44	120,25	$+1{,}75$	42,9%
GRU	162,26	125,56	$+17{,}59$	39,4%
Ingênuo	168,05	126,83	$+0{,}56$	45,0%
Holt-Winters	173,68	133,44	$-1{,}13$	51,5%
Média móvel	187,65	142,68	$+3{,}01$	46,5%
Sazonal ingênuo	360,90	291,42	$-166{,}41$	66,9%

Fonte: Elaborado pelo autor.

A Figura 4.2 resume o custo de cada política no seu ponto de operação, detalhado na Tabela 4.2. O principal resultado é que, entre as políticas que cumprem o limite no teste, ajustar a capacidade à demanda reduz o custo em 24% a 30% com $\varepsilon = 1\%$ e em 32% a 36% com $\varepsilon = 5\%$ em relação ao dimensionamento estático, faixa comparável à de Calheiros et al. (2015). Esse ganho vem da elasticidade, e não da previsão: em $\varepsilon = 5\%$, a melhor preditiva, com o ARIMA, fica só 1,7% abaixo da reativa, e as diferenças entre previsores são de poucos por cento, com a GRU e o ARIMA alternando a primeira posição entre os dois limites. O custo acima do oráculo é quase todo capacidade ociosa, já que a demanda não atendida é desprezível em quase todas as políticas (Tabela 4.3).

Figura 4.2 – Custo de cada política no ponto de operação



Fonte: Elaborado pelo autor.

A ordenação por erro também não se repete na ordenação por custo. A GRU tem RMSE 3% maior que o do ARIMA, mas custa menos em $\varepsilon = 1\%$, coerente com seu viés positivo e sua menor taxa de subestimação, que pedem margem menor (0,28 contra 0,32). O Holt-Winters é o previsor mais caro em $\varepsilon = 1\%$, em parte porque viola menos (V de 0,43% contra 0,87%, dois intervalos), e um dos mais baratos em $\varepsilon = 5\%$. Já o sazonal ingênuo, de pior erro, aparece como o mais barato só porque ultrapassa ε no teste: a demanda muda de comportamento entre a validação e o teste, e a margem escolhida na validação deixa de cobrir a subestimação. A reativa em $\varepsilon = 1\%$ também passa do limite, por um único intervalo.

Por fim, a política reativa altera o número de instâncias com menos da metade da frequência das preditivas, vantagem que não entra no custo. Com antecedência $L = 2$, a vantagem da previsão sobre a reativa diminui: entre as preditivas que cumprem o limite, só a GRU fica abaixo dela em $\varepsilon = 1\%$, 1,5% a menos, e nenhuma em $\varepsilon = 5\%$. Isso contraria a motivação de que o atraso prejudica mais a reativa; hipóteses a testar em OE7 são o RMSE de 10% a 25% maior com $h = 2$ (Tabela A.2) e a janela de estabilização da reativa.

Tabela 4.2 – Pontos de operação no teste

	$\varepsilon = 1\%$					$\varepsilon = 5\%$				
	$L = 1$			$L = 2$		$L = 1$			$L = 2$	
Política	**Par.**	C_{norm}	V (%)	C_{norm}	V (%)	**Par.**	C_{norm}	V (%)	C_{norm}	V (%)
Oráculo	–	1,000	0,00	1,000	0,00	–	1,000	0,00	1,000	0,00
Dimensionamento estático	–	1,830	0,22	1,830	0,22	–	1,830	0,22	1,830	0,22
Reativa (HPA)	0,82	1,340	**1,08**	1,376	0,65	0,92	1,197	4,98	1,223	4,98
Preditiva com:										
ARIMA	0,32	1,315	0,87	1,385	0,65	0,18	1,176	3,90	1,238	3,25
GRU	0,28	1,287	0,87	1,356	0,43	0,19	1,199	3,25	1,226	3,68
Holt-Winters	0,40	1,391	0,43	1,440	0,87	0,19	1,185	3,46	1,233	3,03
Ingênuo	0,38	1,373	0,87	1,412	0,65	0,21	1,206	3,46	1,275	3,90
Média móvel	0,36	1,354	0,43	1,434	0,43	0,24	1,238	3,90	1,238	3,90
Sazonal ingênuo	0,42	1,270	**13,64**	1,270	**13,64**	0,23	1,103	**33,55**	1,103	**33,55**

Fonte: Elaborado pelo autor.

Nota: Par. é a margem m na política preditiva e a utilização-alvo u^* na reativa, escolhida na validação (Seção 3.1) com $L = 1$; com $L = 2$, a escolha é refeita da mesma forma. Valores de V em negrito ultrapassam ε no teste.

Tabela 4.3 – Demanda não atendida, ociosidade e alterações no teste

	$\varepsilon = 1\%$			$\varepsilon = 5\%$		
Política	U (%)	O (%)	A	U (%)	O (%)	A
Oráculo	0,00	1,89	378	0,00	1,89	378
Dimensionamento estático	0,01	46,41	0	0,01	46,41	0
Reativa (HPA)	0,06	26,84	116	0,39	18,37	116
Preditiva com:						
ARIMA	0,04	25,40	386	0,29	16,84	379
GRU	0,06	23,83	345	0,24	18,37	326
Holt-Winters	0,07	29,50	409	0,31	17,49	411
Ingênuo	0,02	28,57	403	0,25	18,82	397
Média móvel	0,06	27,60	284	0,26	20,92	282
Sazonal ingênuo	1,22	23,66	408	4,34	14,89	398

Fonte: Elaborado pelo autor.

Nota: valores nos pontos de operação da Tabela 4.2; o oráculo e o dimensionamento estático não dependem de ε.

5
Cronogramas

Nas duas tabelas, as células marcam os meses de cada atividade, e a coluna **OE** a relaciona aos objetivos da Tabela 3.3.

5.1
Projeto Final I

O Projeto Final I ocupou de agosto a novembro de 2026 (Tabela 5.1): agosto e setembro foram dedicados à escolha do tema, aos estudos e à proposta, e outubro, à construção e à avaliação do protótipo.

Tabela 5.1 – Cronograma do Projeto Final I

Atividade	OE	Ago	Set	Out	Nov	Conclusão
Escolha do tema e da pergunta	–					–
Revisão bibliográfica	–					–
Elaboração da proposta	–					–
Ambiente e repositório	OE10					14/10
Tratamento dos dados	OE1					15/10
Previsores tradicionais	OE2					15/10
Simulador e políticas	OE3					15/10
Rede neural (GRU)	OE2					20/10
Experimentos preliminares	OE5 a OE7					21/10
Redação do relatório	–					–
Entrega do relatório	–					22/11 (prevista)

Fonte: Elaborado pelo autor.

5.2
Projeto Final II

O Projeto Final II vai de dezembro de 2026 à defesa, no fim de junho de 2027 (Tabela 5.2), e começa pela memória, de que dependem todos os resultados finais.

Tabela 5.2 – Cronograma do Projeto Final II

		2026	2027					
Atividade	OE	Dez	Jan	Fev	Mar	Abr	Mai	Jun
Incorporação da memória	OE4							
Replicação no rastro do Google	OE9							
Várias sementes e janelas de teste	OE8							
Experimentos finais	OE5, OE6							
Sensibilidade à antecedência L	OE7							
Opcional: quantil previsto pela rede	–							
Opcional: validação no CloudSim Plus	–							
Manutenção do repositório	OE10							
Redação do relatório final	–							
Apresentação e defesa	–							

Fonte: Elaborado pelo autor.

Referências

AGULLÓ, F. et al. Enhancing the output of time series forecasting algorithms for cloud resource provisioning. **Future Generation Computer Systems**, v. 170, p. 107833, 2025.

ALIBABA GROUP. **Alibaba Cluster Trace Program**. 2018. Disponível em: https://github.com/alibaba/clusterdata. Acesso em: 14 out. 2026.

AMAZON WEB SERVICES. **Predictive scaling for Amazon EC2 Auto Scaling**. 2024. Documentação do Amazon EC2 Auto Scaling. Disponível em: https://docs.aws.amazon.com/autoscaling/ec2/userguide/ec2-auto-scaling-predictive-scaling.html. Acesso em: 14 out. 2026.

BELOGLAZOV, A.; BUYYA, R. Optimal online deterministic algorithms and adaptive heuristics for energy and performance efficient dynamic consolidation of virtual machines in cloud data centers. **Concurrency and Computation: Practice and Experience**, v. 24, n. 13, p. 1397–1420, 2012.

BENGIO, Y.; SIMARD, P.; FRASCONI, P. Learning long-term dependencies with gradient descent is difficult. **IEEE Transactions on Neural Networks**, v. 5, n. 2, p. 157–166, 1994.

BEN TAIEB, S. et al. A review and comparison of strategies for multi-step ahead time series forecasting based on the NN5 forecasting competition. **Expert Systems with Applications**, v. 39, n. 8, p. 7067–7083, 2012.

BEYER, B. et al. **Site Reliability Engineering: How Google Runs Production Systems**. Sebastopol, CA: O'Reilly Media, 2016.

BODRA, D.; KHAIRNAR, S. Machine learning-based cloud resource allocation algorithms: a comprehensive comparative review. **Frontiers in Computer Science**, v. 7, p. 1678976, 2025.

BOX, G. E. P. et al. **Time Series Analysis: Forecasting and Control**. 5. ed. Hoboken, NJ: John Wiley & Sons, 2015.

CALHEIROS, R. N. et al. Workload prediction using ARIMA model and its impact on cloud applications' QoS. **IEEE Transactions on Cloud Computing**, v. 3, n. 4, p. 449–458, 2015.

CALHEIROS, R. N. et al. CloudSim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms. **Software: Practice and Experience**, v. 41, n. 1, p. 23–50, 2011.

CHO, K. et al. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In: **Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)**. [S.l.: s.n.], 2014. p. 1724–1734.

CHUNG, J. et al. Empirical evaluation of gated recurrent neural networks on sequence modeling. In: **NIPS 2014 Workshop on Deep Learning**. [S.l.: s.n.], 2014. Disponível em: https://arxiv.org/abs/1412.3555. Acesso em: 14 out. 2026.

DIEBOLD, F. X.; MARIANO, R. S. Comparing predictive accuracy. **Journal of Business & Economic Statistics**, v. 13, n. 3, p. 253–263, 1995.

EFRON, B.; TIBSHIRANI, R. J. **An Introduction to the Bootstrap**. New York: Chapman & Hall, 1993.

HARRIS, C. R. et al. Array programming with NumPy. **Nature**, v. 585, p. 357–362, 2020.

HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. **Neural Computation**, v. 9, n. 8, p. 1735–1780, 1997.

HOLM, S. A simple sequentially rejective multiple test procedure. **Scandinavian Journal of Statistics**, v. 6, n. 2, p. 65–70, 1979.

HUNTER, J. D. Matplotlib: A 2D graphics environment. **Computing in Science & Engineering**, v. 9, n. 3, p. 90–95, 2007.

HYNDMAN, R. J.; ATHANASOPOULOS, G. **Forecasting: principles and practice**. 2. ed. Melbourne: OTexts, 2018. Disponível em: https://otexts.com/fpp2/. Acesso em: 14 out. 2026.

LORIDO-BOTRAN, T.; MIGUEL-ALONSO, J.; LOZANO, J. A. A review of auto-scaling techniques for elastic applications in cloud environments. **Journal of Grid Computing**, v. 12, n. 4, p. 559–592, 2014.

MAO, M.; HUMPHREY, M. A performance study on the VM startup time in the cloud. In: **2012 IEEE Fifth International Conference on Cloud Computing (CLOUD)**. [S.l.: s.n.], 2012. p. 423–430.

MCKINNEY, W. Data structures for statistical computing in Python. In: WALT, S. van der; MILLMAN, J. (Ed.). **Proceedings of the 9th Python in Science Conference**. [S.l.: s.n.], 2010. p. 56–61.

MELL, P.; GRANCE, T. **The NIST Definition of Cloud Computing**. Gaithersburg, MD: National Institute of Standards and Technology, 2011. (NIST Special Publication, 800-145).

PAN, Z. et al. MagicScaler: Uncertainty-aware, predictive autoscaling. **Proceedings of the VLDB Endowment**, v. 16, n. 12, p. 3808–3821, 2023.

PASZKE, A. et al. PyTorch: An imperative style, high-performance deep learning library. In: **Advances in Neural Information Processing Systems 32 (NeurIPS 2019)**. [S.l.: s.n.], 2019.

RAASVELDT, M.; MÜHLEISEN, H. DuckDB: an embeddable analytical database. In: **Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19)**. [S.l.]: ACM, 2019. p. 1981–1984.

ROSAS, G. L. de S. **predictive-cloud-autoscaling: código do Projeto Final**. 2026. Disponível em: https://github.com/gabriellsar/predictive-cloud-autoscaling. Acesso em: 20 out. 2026.

ROSSI, A. et al. Forecasting workload in cloud computing: towards uncertainty-aware predictions and transfer learning. **Cluster Computing**, v. 28, n. 4, p. 258, 2025. Código e dados disponíveis em: https://github.com/andreareds/TowardsUncertaintyAwareWorkloadPrediction. Acesso em: 14 out. 2026.

SEABOLD, S.; PERKTOLD, J. statsmodels: Econometric and statistical modeling with Python. In: **Proceedings of the 9th Python in Science Conference**. [S.l.: s.n.], 2010.

SILVA FILHO, M. C. et al. CloudSim Plus: a cloud computing simulation framework pursuing software engineering principles for improved modularity, extensibility and correctness. In: **2017 IFIP/IEEE Symposium on Integrated Network and Service Management (IM)**. [S.l.: s.n.], 2017. p. 400–406.

THE KUBERNETES AUTHORS. **Horizontal Pod Autoscaling**. 2024. Documentação do Kubernetes. Disponível em: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/. Acesso em: 14 out. 2026.

WILKES, J. **Google cluster-usage traces v3**. Mountain View, CA, USA, 2020. Disponível em: https://github.com/google/cluster-data. Acesso em: 14 out. 2026.

XU, M. et al. esDNN: Deep neural network based multivariate workload prediction in cloud computing environments. **ACM Transactions on Internet Technology**, v. 22, n. 3, p. 75:1–75:24, 2022.

A
Erro de previsão por horizonte

As Tabelas A.1 e A.2 complementam a Tabela 4.1 com o erro dos seis previsores nos horizontes $h = 1$, 2, 3 e 6 (OE2), na validação e no teste. As tabelas são geradas pelo *pipeline* a partir de `results/tables/forecast_metrics.csv`.

Tabela A.1 – Erro das previsões na validação, por horizonte

Previsor	h	**RMSE**	**MAE**	**MAPE (%)**	**Viés**	**Taxa de subestimação**
ARIMA	1	164,10	122,91	8,35	$-4{,}51$	47,4%
	2	199,70	146,58	9,87	$-8{,}04$	47,0%
	3	216,01	158,14	10,58	$-10{,}65$	48,0%
	6	245,61	174,89	11,62	$-16{,}76$	49,4%
GRU	1	167,32	126,06	8,77	$+10{,}11$	39,6%
	2	198,43	150,03	10,44	$+8{,}66$	40,0%
	3	213,01	160,85	11,22	$+10{,}21$	40,0%
	6	239,02	177,00	12,35	$+2{,}38$	42,4%
Ingênuo	1	174,68	131,38	8,97	$-1{,}38$	48,3%
	2	219,60	162,99	10,99	$-3{,}29$	47,8%
	3	238,43	174,84	11,72	$-5{,}30$	49,6%
	6	270,07	188,58	12,63	$-11{,}72$	50,4%
Holt-Winters	1	178,97	132,96	9,19	$+0{,}72$	51,1%
	2	220,68	161,36	11,16	$+1{,}06$	50,9%
	3	236,05	172,30	11,96	$+1{,}73$	51,3%
	6	235,93	177,68	12,42	$+3{,}92$	51,3%
Média móvel	1	194,53	142,77	9,62	$-6{,}29$	48,9%
	2	213,92	156,22	10,49	$-8{,}22$	49,4%
	3	227,35	164,91	11,06	$-10{,}02$	50,6%
	6	253,98	180,73	11,94	$-14{,}78$	49,4%
Sazonal ingênuo	1	295,52	222,57	15,89	$+96{,}08$	34,4%
	2	295,52	222,57	15,89	$+96{,}08$	34,4%
	3	295,52	222,57	15,89	$+96{,}08$	34,4%
	6	295,52	222,57	15,89	$+96{,}08$	34,4%

Fonte: Elaborado pelo autor.

Nota: RMSE, MAE e viés em máquinas equivalentes; viés positivo indica superestimação. Cada linha usa as mesmas 460 origens, e a máscara do MAPE não excluiu nenhuma. O sazonal ingênuo repete o valor de um dia antes e, por isso, não depende de h.

Tabela A.2 – Erro das previsões no teste, por horizonte

Previsor	h	**RMSE**	**MAE**	**MAPE (%)**	**Viés**	**Taxa de subestimação**
ARIMA	1	157,44	120,25	7,45	$+1{,}75$	42,9%
	2	190,36	143,76	8,95	$+3{,}15$	45,7%
	3	206,16	157,83	9,88	$+4{,}24$	45,5%
	6	234,88	178,64	11,13	$+6{,}15$	45,9%
GRU	1	162,26	125,56	7,89	$+17{,}59$	39,4%
	2	191,25	146,67	9,27	$+15{,}31$	43,1%
	3	204,36	157,55	9,96	$+13{,}08$	43,1%
	6	229,60	173,90	10,88	$-4{,}40$	47,6%
Ingênuo	1	168,05	126,83	7,81	$+0{,}56$	45,0%
	2	210,23	157,53	9,74	$+1{,}45$	46,5%
	3	229,81	176,29	10,98	$+2{,}68$	46,5%
	6	267,34	203,26	12,71	$+5{,}65$	47,0%
Holt-Winters	1	173,68	133,44	8,24	$-1{,}13$	51,5%
	2	209,23	160,20	9,99	$-1{,}54$	50,9%
	3	221,27	172,81	10,79	$-1{,}92$	50,9%
	6	255,43	199,16	12,42	$-4{,}13$	50,6%
Média móvel	1	187,65	142,68	8,90	$+3{,}01$	46,5%
	2	207,27	158,16	9,88	$+3{,}86$	46,1%
	3	219,73	168,03	10,50	$+4{,}54$	46,5%
	6	242,20	186,24	11,55	$+5{,}37$	46,5%
Sazonal ingênuo	1	360,90	291,42	17,56	$-166{,}41$	66,9%
	2	360,90	291,42	17,56	$-166{,}41$	66,9%
	3	360,90	291,42	17,56	$-166{,}41$	66,9%
	6	360,90	291,42	17,56	$-166{,}41$	66,9%

Fonte: Elaborado pelo autor.

Nota: RMSE, MAE e viés em máquinas equivalentes; viés positivo indica superestimação. Cada linha usa as mesmas 462 origens, e a máscara do MAPE não excluiu nenhuma. O sazonal ingênuo repete o valor de um dia antes e, por isso, não depende de h.

B
Parâmetros dos dados, dos previsores e do Projeto Final II

A Tabela B.1 complementa a Tabela 3.1 com os parâmetros dos dados, dos previsores e do Projeto Final II.

Tabela B.1 – Parâmetros dos dados, dos previsores e do Projeto Final II

Parâmetro	Símbolo	Valor
Dados		
Cobertura mínima por intervalo	–	50% das máquinas; abaixo disso, o intervalo é lacuna
Lacunas curtas	–	Até 3 intervalos, interpolação linear
Lacunas longas	–	Mesmo horário do dia anterior, ajustado às bordas, só dentro do treino
Trecho inicial do rastro	–	Mantido, apesar da cobertura menor
Precisão da série gravada	–	6 casas decimais
Previsores tradicionais		
Janela da média móvel	w	6 intervalos (30 min)
Período sazonal	s	288 intervalos (1 dia), no sazonal ingênuo e no Holt-Winters
Holt-Winters	–	Tendência e sazonalidade aditivas, inicialização heurística; α, β e γ estimados no treino e mantidos fixos
ARIMA	(p, d, q)	$(2, 1, 2)$, fixada a priori, sem busca de ordens; coeficientes estimados no treino e mantidos fixos; resíduos do treino sem autocorrelação até a defasagem 12 (Ljung-Box)
Limiar da máscara do MAPE	–	1% da mediana do treino
GRU		
Janela de entrada	w	72 intervalos (6 h)
Arquitetura	–	1 camada de 32 unidades e saída linear para $h = 1$ a 6 (estratégia direta), sem busca de hiperparâmetros
Treinamento	–	Perda MSE, Adam com taxa 0,001, lotes de 32, até 100 épocas, precisão dupla
Parada antecipada	–	Paciência de 10 épocas, nos últimos 15% das janelas do treino, separados do ajuste por 6 janelas; restaura a melhor época
Normalização	–	Média e desvio padrão do treino
Semente (PF I)	–	42
Projeto Final II		
Sementes da GRU	–	10, de 42 a 51
Janelas de teste na Alibaba	–	3 de 1 dia (288 intervalos); treino expansivo; validação de 1 dia
Janelas de teste no Google	–	3 a 4 de 2 a 3 dias por célula; treino expansivo de pelo menos 14 dias; validação do tamanho do teste

Fonte: Elaborado pelo autor.